

Лабораторная работа №2

Отчёт

Коровкин Никита Михайлович

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
4	Выводы	20
5	Ответы на контрольные вопросы	21
	Список литературы	23

Список иллюстраций

3.1	Установка git	7
3.2	Установка gh	8
3.3	Указание имени	9
3.4	Указание почты и настройка кодировки utf8	10
3.5	Настройка параметров git	11
3.6	Создание SSH ключа	12
3.7	Создание gpg ключа	13
3.8	Вводим пароли	14
3.9	Авторизируемся на GitHub	15
3.10	Авторизация прошла успешно	16
3.11	Настройка автоматических подписей коммитов	17
3.12	Фото репозитория	18
3.13	Фото репозитория	19

Список таблиц

1 Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.[1]

2 Задание

Создать базовую конфигурацию для работы с git.

Создать ключ SSH.

Создать ключ PGP.

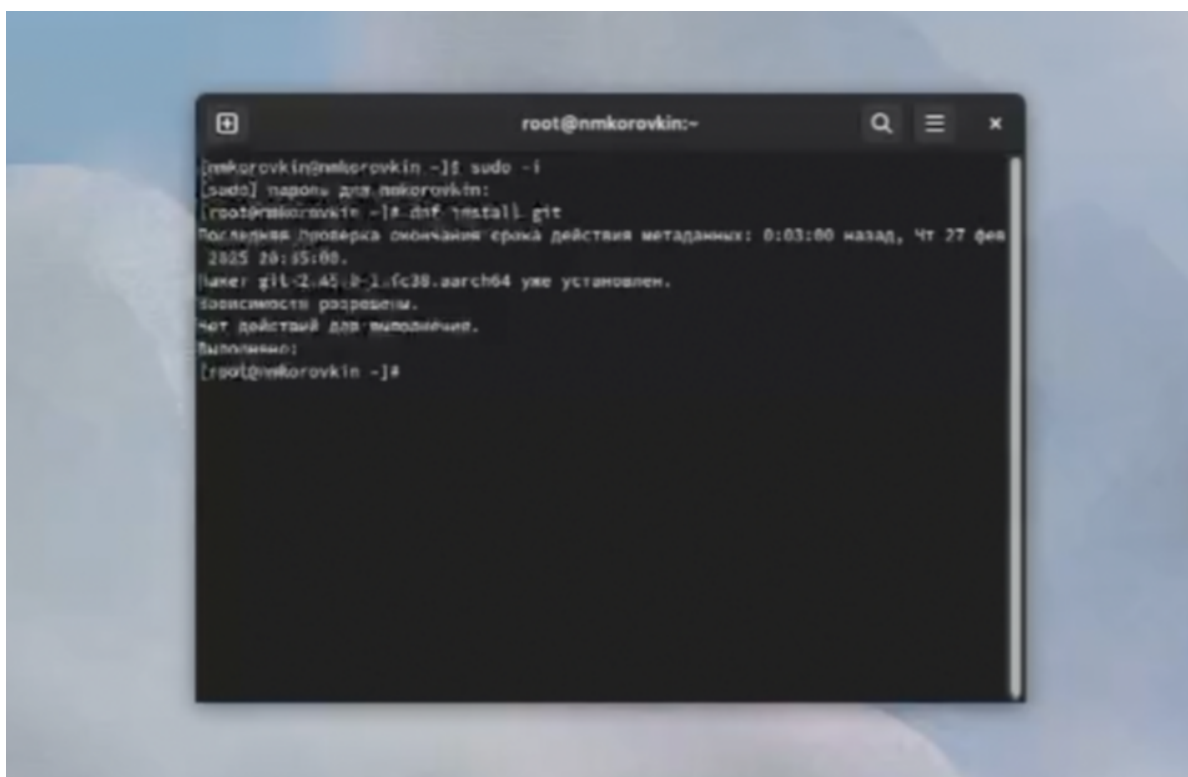
Настроить подписи git.

Зарегистрироваться на Github.

Создать локальный каталог для выполнения заданий по предмету.

3 Выполнение лабораторной работы

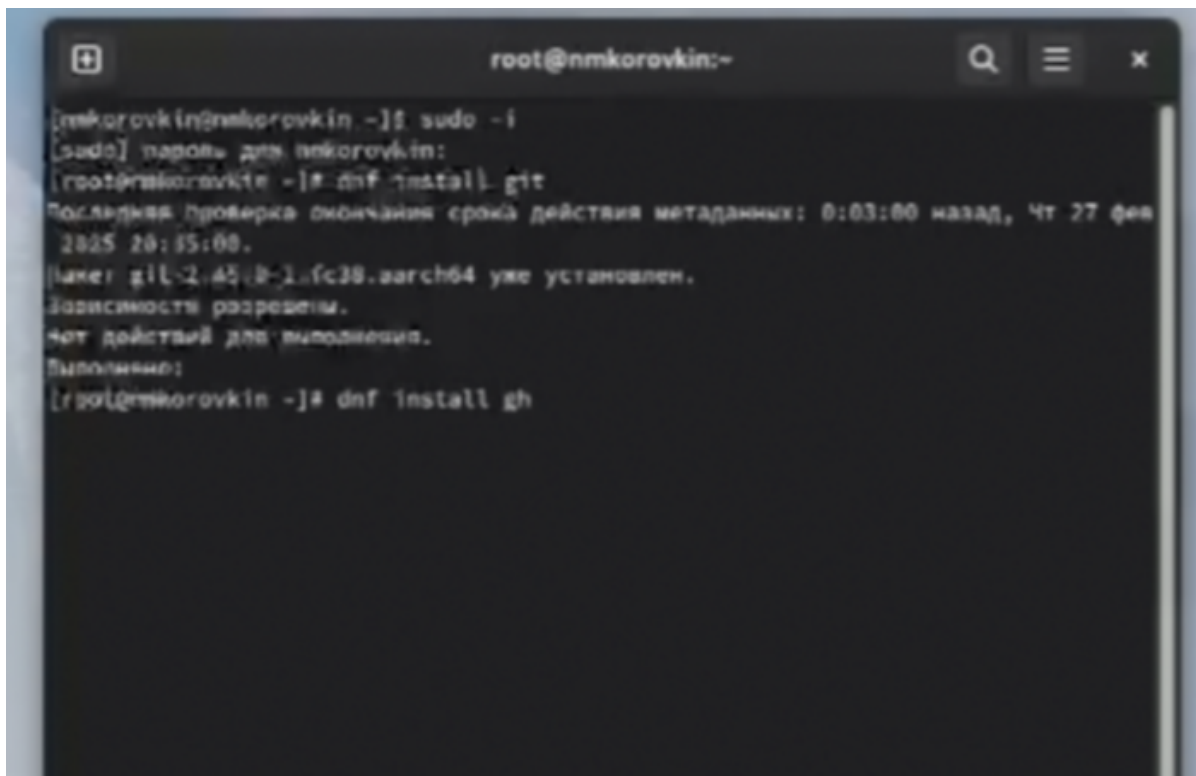
Для начала установим git. Я установил его заранее, поэтому проверим его наличие и версию.(рис. 3.1)

A photograph of a terminal window on a laptop screen. The terminal has a dark background with white text. The window title is 'root@nmkorovkin:~'. The user is in the 'nmkorovkin' directory. They run 'sudo -i' to become root. Then they enter the password 'nmkorovkin'. Next, they run 'dpkg --get-architecture' which returns 'amd64'. Then they run 'dpkg-query -f='\${Package} \${Version} \${Architecture}\n' -W git' which returns 'git 2.45.0-1.fc38.amd64'. Finally, they run 'dpkg-query -f='\${Package} \${Version} \${Architecture}\n' -W git' which returns 'git 2.45.0-1.fc38.amd64'. The terminal output shows the following:

```
nmkorovkin@nmkorovkin ~]$ sudo -i
[sudo] пароль для nmkorovkin:
nmkorovkin@nmkorovkin ~]$ dpkg --get-architecture
amd64
nmkorovkin@nmkorovkin ~]$ dpkg-query -f='${Package} ${Version} ${Architecture}\n' -W git
git 2.45.0-1.fc38.amd64
nmkorovkin@nmkorovkin ~]$ dpkg-query -f='${Package} ${Version} ${Architecture}\n' -W git
git 2.45.0-1.fc38.amd64
```

Рис. 3.1: Установка git

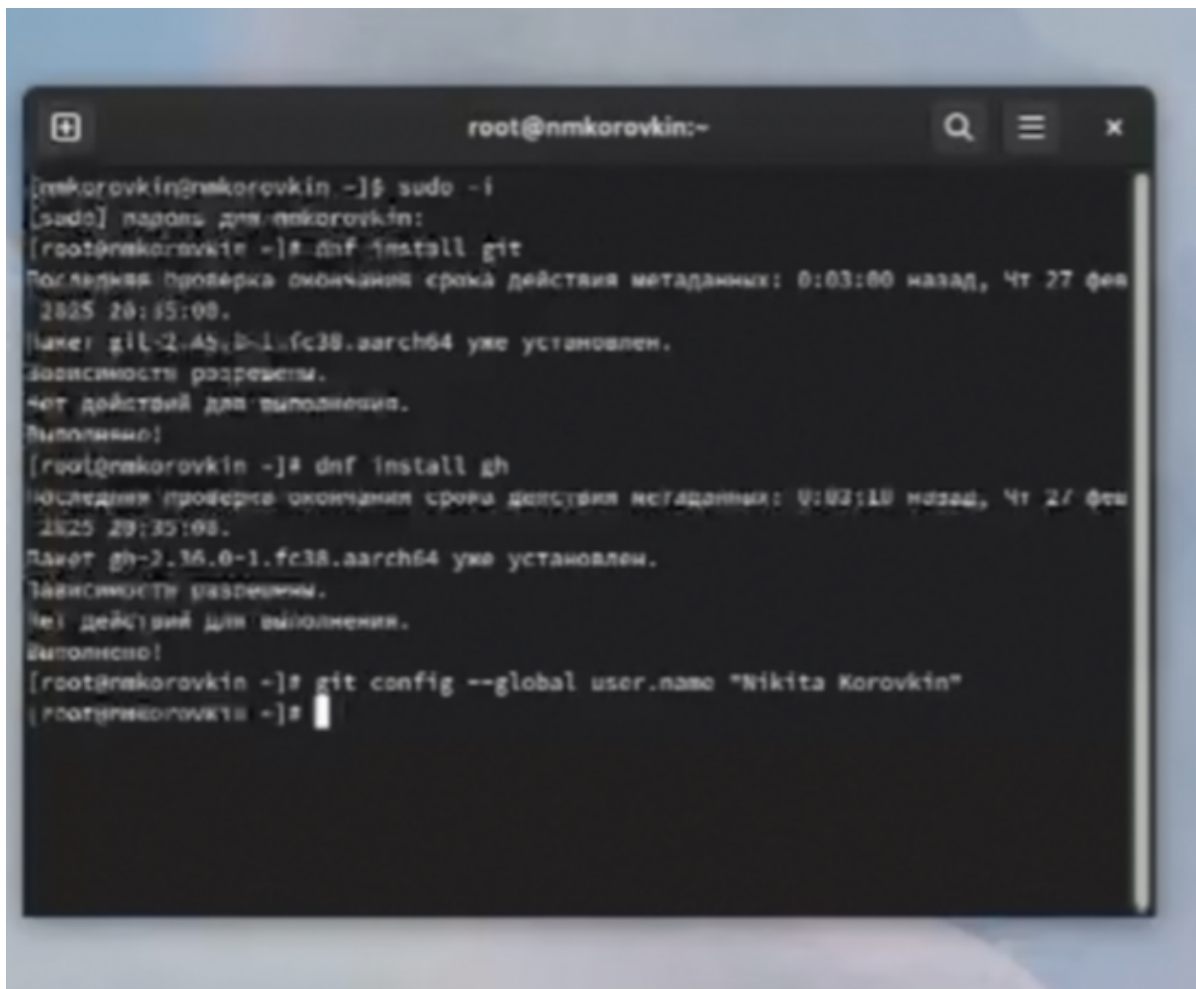
Теперь установим gh или проверим его наличие (рис. 3.2)



```
root@nmkorovkin:~  
[nmkorovkin@nmkorovkin ~]$ sudo -i  
[sudo] пароль для nmkorovkin:  
[root@nmkorovkin ~]# dnf install git  
Последняя проверка окончания срока действия метаданных: 0:03:00 назад, Чт 27 фев 2025 20:35:00.  
[make: git-2.45.0-1.fc38.aarch64 уже установлен.  
Зависимости разрешены.  
Чет действий для выполнения.  
Выполнено:  
[root@nmkorovkin ~]# dnf install gh
```

Рис. 3.2: Установка gh

Теперь установим имя владельца репозитория. Я использую свое имя (рис. 3.3)



```
root@nmkorovkin:~  
[nmkorovkin@nmkorovkin ~]$ sudo -i  
[sudo] пароль для nmkorovkin:  
[root@nmkorovkin ~]$ dnf install git  
Последняя проверка окончания срока действия метаданных: 0:03:00 назад, Чт 27 фев 2025 20:15:00.  
Пакет git-2.45.0-1.fc38.aarch64 уже установлен.  
Зависимости разрешены.  
Нет действий для выполнения.  
Выполнено!  
[root@nmkorovkin ~]$ dnf install gh  
Последняя проверка окончания срока действия метаданных: 0:03:10 назад, Чт 27 фев 2025 20:15:00.  
Пакет gh-2.36.0-1.fc38.aarch64 уже установлен.  
Зависимости разрешены.  
Нет действий для выполнения.  
Выполнено!  
[root@nmkorovkin ~]$ git config --global user.name "Nikita Korovkin"  
[root@nmkorovkin ~]$
```

Рис. 3.3: Указание имени

Теперь зададим почту и настроим кодировку utf8 в выводе сообщений git. (рис. 3.4)

```
root@nmkorovkin:~  
[nmkorovkin@nmkorovkin ~]$ sudo -i  
[sudo] пароль для nmkorovkin:  
[root@nmkorovkin ~]# dnf install git  
Последняя проверка окончания срока действия метаданных: 0:03:00 назад, Чт 27 фев  
2025 20:35:00.  
Пакет git-2.45.0-1.fc38.aarch64 уже установлен.  
Зависимости разрешены.  
Чет действий для выполнения.  
Выполнено!  
[root@nmkorovkin ~]# dnf install gh  
Последняя проверка окончания срока действия метаданных: 0:02:10 назад, Чт 27 фев  
2025 20:35:00.  
Пакет gh-2.36.0-1.fc38.aarch64 уже установлен.  
Зависимости разрешены.  
Чет действий для выполнения.  
Выполнено!  
[root@nmkorovkin ~]# git config --global user.name "Nikita Korovkin"  
[root@nmkorovkin ~]# git config --global user.email "jky.world.y.t@gmail.com"  
[root@nmkorovkin ~]# git config --global core.quotepath false  
[root@nmkorovkin ~]#
```

Рис. 3.4: Указание почты и настройка кодировки utf8

Зададим имя начальной ветки, настроим параметры autocrlf и safecrlf (рис. 3.5)

```
Последняя проверка окончания срока действия метаданных: 0:02:10 назад, Чт 27 фев 2025 20:35:00.  
Пакет git-2.45.0-1.fc38.aarch64 уже установлен.  
Зависимости разрешены.  
Нет действий для выполнения.  
Выполнено!  
[root@nikorovkin ~]# dnf install gh  
Последняя проверка окончания срока действия метаданных: 0:02:10 назад, Чт 27 фев 2025 20:35:00.  
Пакет gh-2.36.0-1.fc38.aarch64 уже установлен.  
Зависимости разрешены.  
Нет действий для выполнения.  
Выполнено!  
[root@nikorovkin ~]# git config --global user.name "Nikita Korovkin"  
[root@nikorovkin ~]# git config --global user.email "jky.world.y.t@gmail.com"  
[root@nikorovkin ~]# git config --global core.quietpath false  
[root@nikorovkin ~]# git config --global init.defaultBranch master  
[root@nikorovkin ~]# git config --global core.autocrlf input  
[root@nikorovkin ~]# git config --global core.safecrlf warn  
[root@nikorovkin ~]#
```

Рис. 3.5: Настройка параметров git

Далее создадим SSH ключ для подключения.(рис. 3.6)

```
[root@mkarovkin ~]# git config --global core.safecrlf warn
[root@mkarovkin ~]# ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa): key
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in key
Your public key has been saved in key.pub
The key fingerprint is:
SHA256: /Aa1Lq114H2x10A1ZAXLc3Hk//6GdyopSV4h0No+44 root@mkarovkin
The key's randomart image is:
+--[RSA 4096]--+
|
| .
| + o
| + B .
| o. + S
| +.oB B .
| =+ .B + . .
| +2==+ .o o .
| +E--+BBo. .
+-----[SHA256]-----
[root@mkarovkin ~]#
```

Рис. 3.6: Создание SSH ключа

Теперь нам нужно создать gpg ключи. Выбираем из предложенных вариантов первый тип (RSA and RSA), размер ключа задаём 4096 бит и делаем срок действия ключа неограниченным (рис. 3.7)

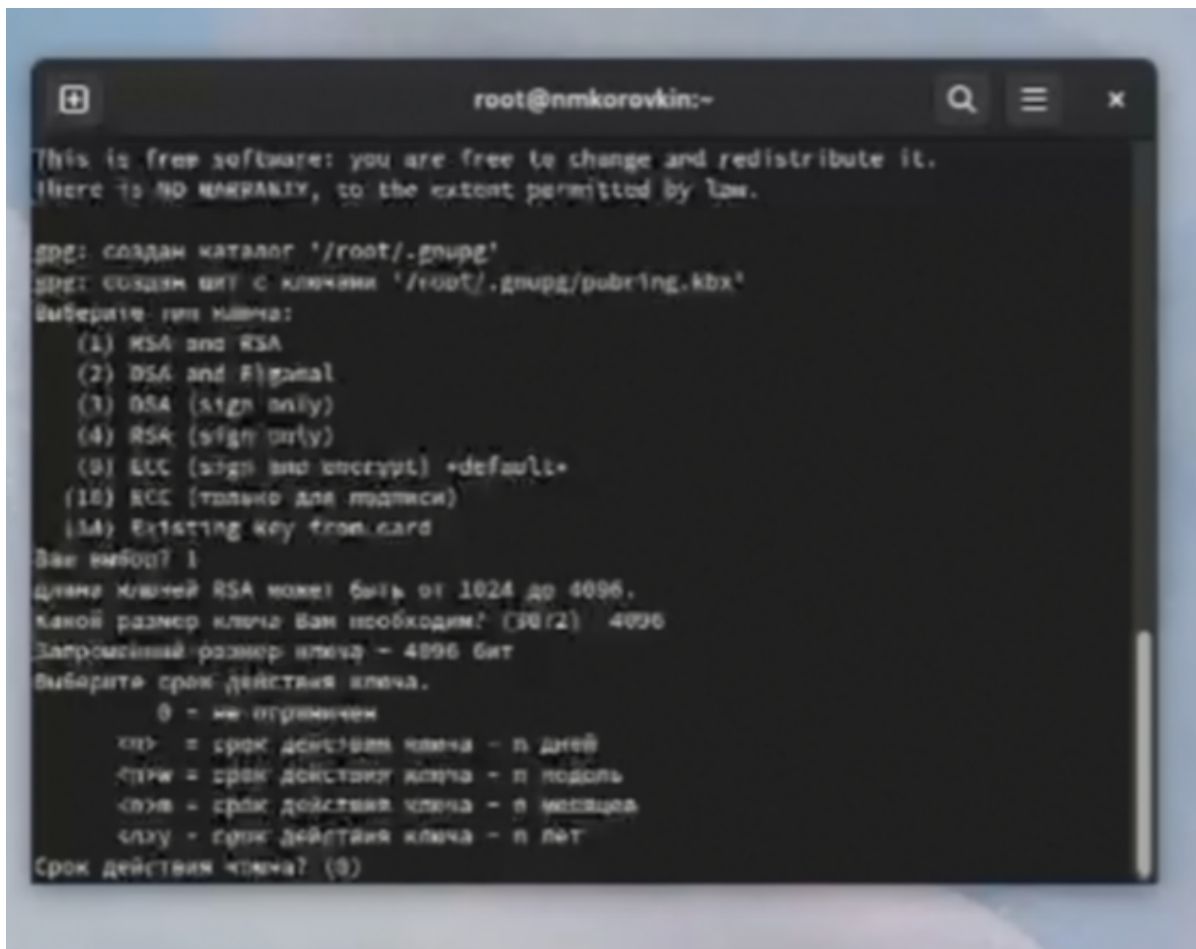


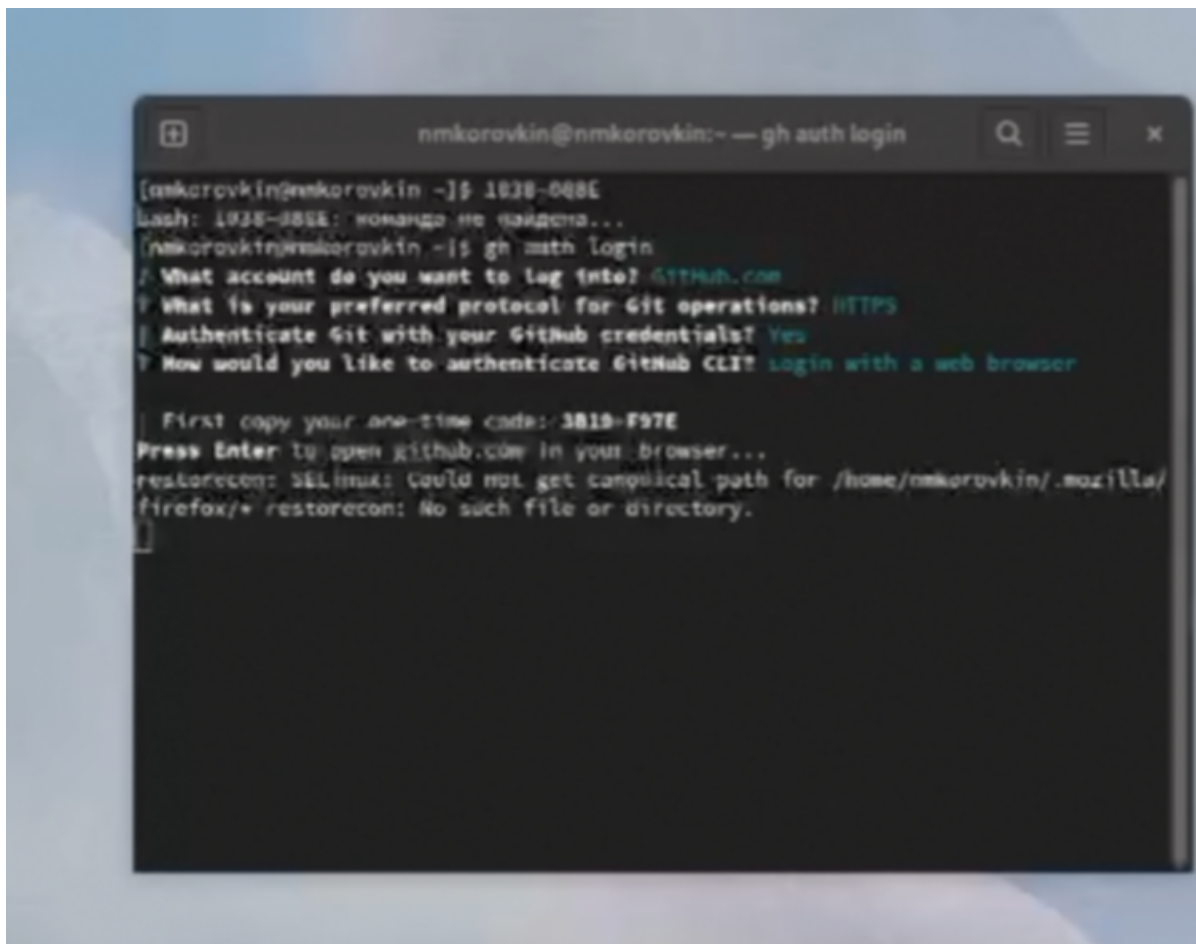
Рис. 3.7: Создание gpg ключа

После введение Имени и почты, введем пароль для защиты наших ключей.(рис. 3.8)



Рис. 3.8: Вводим пароли

Затем авторизируемся на GitHub любым способом. Я буду делать это через браузер(рис. 3.9)

A terminal window titled 'nmkorovkin@nmkorovkin:~ — gh auth login'. The user enters '1838-088E' and 'gh auth login'. The terminal displays a series of prompts: 'What account do you want to log into? GitHub.com', 'What is your preferred protocol for Git operations? HTTPS', 'Authenticate Git with your GitHub credentials? Yes', and 'How would you like to authenticate GitHub CLI? Login with a web browser'. It then shows the one-time code '3819-F97E' and instructs the user to press Enter to open github.com in their browser. At the bottom, there is an error message: 'restorecon: SELinux: Could not get canonical path for /home/nmkorovkin/.mozilla/firefox/* restorecon: No such file or directory.'

```
nmkorovkin@nmkorovkin:~ — gh auth login
[nmkorovkin@nmkorovkin ~]$ 1838-088E
bash: 1838-088E: command not found...
[nmkorovkin@nmkorovkin ~]$ gh auth login
? What account do you want to log into? GitHub.com
? What is your preferred protocol for Git operations? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

First copy your one-time code: 3819-F97E
Press Enter to open github.com in your browser...
restorecon: SELinux: Could not get canonical path for /home/nmkorovkin/.mozilla/
firefox/* restorecon: No such file or directory.
```

Рис. 3.9: Авторизируемся на GitHub

Теперь мы авторизовались на сайте.(рис. 3.10)

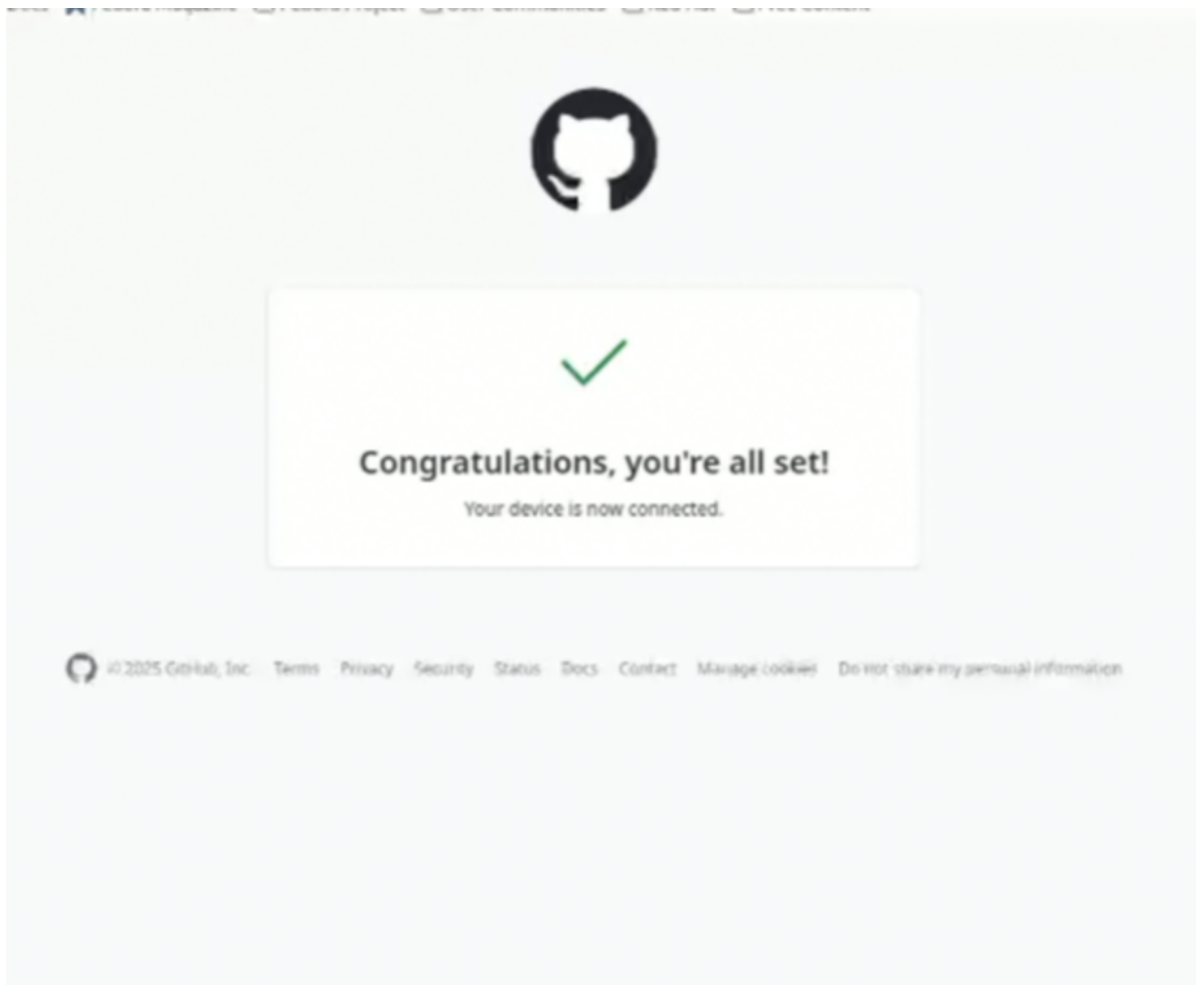
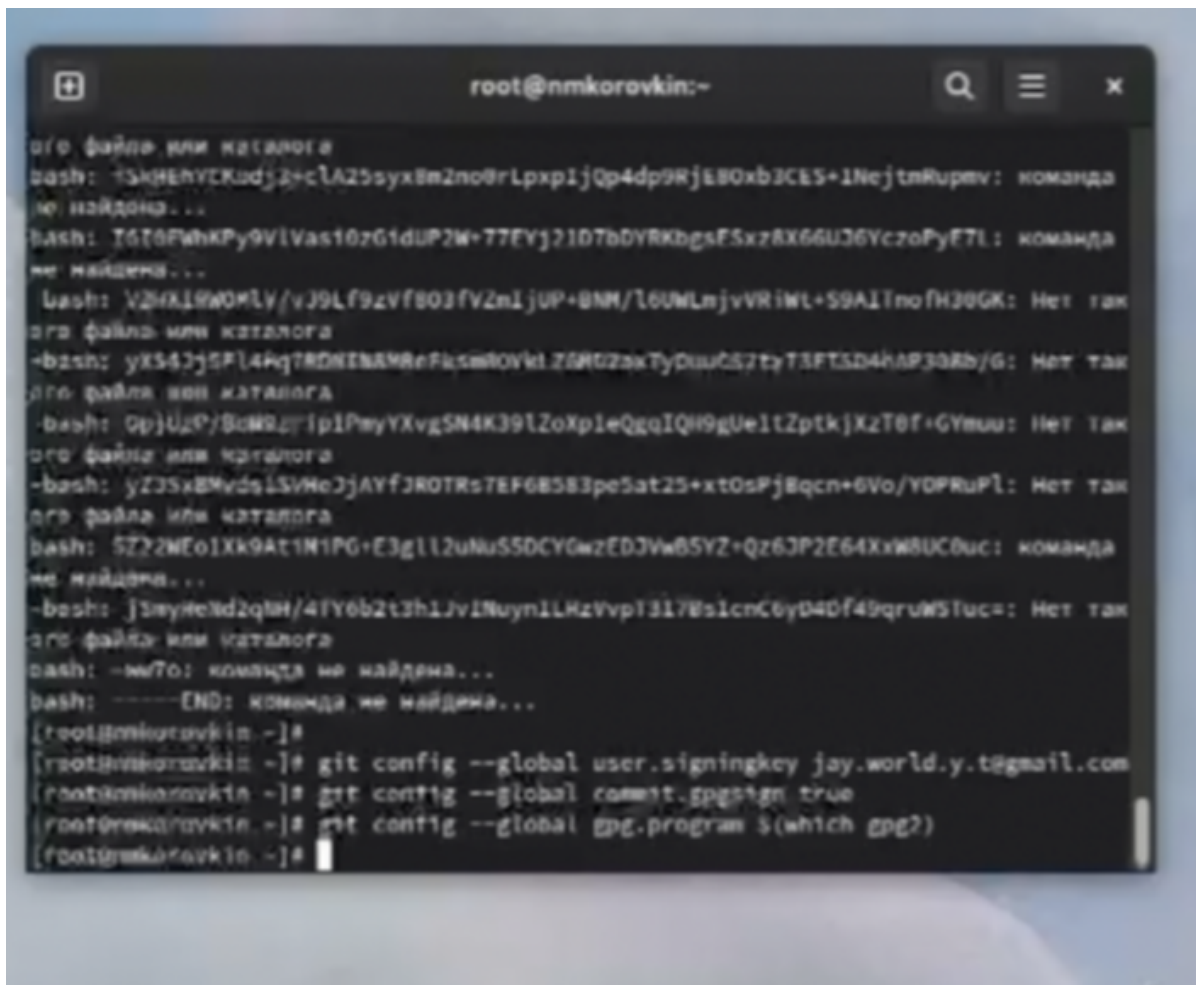


Рис. 3.10: Авторизация прошла успешно

Теперь настроим автоматические подписи коммитов.(рис. 3.11)



```
root@nmkorovkin:~  
bash: 15kHEHYCKudj3+c1A25syx8m2no0rLpxp1jQp4dp9RjL80xb3CE5+1NejtnRupmv: команда  
не найдена...  
bash: 16T6FWHKPy9VlVas16zG1dUP2W+77EVj21D7bDYRKbgsF5xz8X66UJ6Ycz0PyE7L: команда  
не найдена...  
bash: V2hX18WOMlv/vJ9Lf9zVf803fV2m1jUP+8NH/16UWLnjvVRiWt+59A1TnoFH30GK: Нет так  
ого файла или каталога  
-bash: yX543j5Pl4Aq78DNTBAM8eFkxsm8OVklZ8MCZaxTyDuuCS7tyT3FT3D4hAP30Rb/G: Нет так  
ого файла или каталога  
-bash: Qp)UzP/8cW8z: ipiPwyYXvg5N4K39lZoXpleQgqIQH9gUeltZptkjXzT8f+GYuuu: Нет так  
ого файла или каталога  
-bash: yZ35x8Mvds1SVhcJjAYfJROTRs7EF68583pe5at25+xt0sPj8qcn+6Vo/YDPRuPl: Нет так  
ого файла или каталога  
bash: 5Z22NEo1Xk9At1M1PG+E3gl12uNu55DCY0wzEDJvWb5YZ+Qz6JP2E64XxW8UC0uc: команда  
не найдена...  
-bash: j3myNeXd2qH/4fY6b2t3h1jv1Nuyn1LH2Vvp1317BslcnC6yD4Df49gruW5Tuc=: Нет так  
ого файла или каталога  
bash: --wTo: команда не найдена...  
bash: -----END: команда не найдена...  
[root@nmkorovkin ~]#  
[root@nmkorovkin ~]# git config --global user.signingkey jay.world.y.t@gmail.com  
[root@nmkorovkin ~]# git config --global commit.gpgsign true  
[root@nmkorovkin ~]# git config --global gpg.program $(which gpg)  
[root@nmkorovkin ~]#
```

Рис. 3.11: Настройка автоматических подписей коммитов

Далее нам необходимо создать репозиторий и загрузить его на сервер. Этот этап я пропущу, так как репозиторий уже был создан.

Приложу фото репозитория на операционной системе(рис. 3.12)

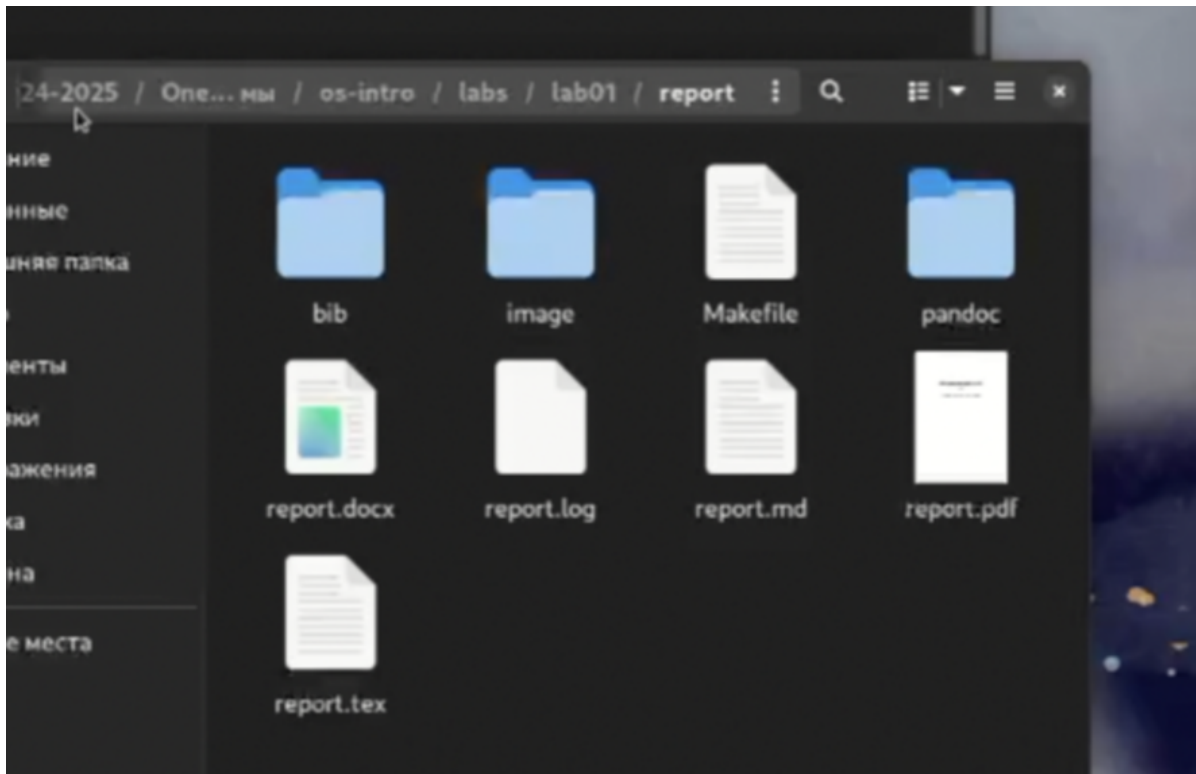


Рис. 3.12: Фото репозитория

Репозиторий на сервере(рис. 3.13)

 Pancakeboy1987	feat(main): added lab01	fdd2729 · 4 days ago	
 config	Initial commit	last week	
 labs	feat(main): added lab01	4 days ago	
 presentation	feat(main): make cour...	5 days ago	
 project-personal	feat(main): make cour...	5 days ago	
 template	Initial commit	last week	
 .gitattributes	Initial commit	last week	
 .gitignore	Initial commit	last week	
 .gitmodules	Initial commit	last week	
 CHANGELOG.md	Initial commit	last week	
 COURSE	feat(main): make cour...	5 days ago	

Рис. 3.13: Фото репозитория

4 Выводы

Была произведена установка git, проведена его первоначальная настройка, были созданы ключи для авторизации и подписи, а также создан репозиторий курса из предложенного шаблона

5 Ответы на контрольные вопросы

1. Системы контроля версий – это системы, в которых мы можем хранить свои проекты и выкладывать их обновления, контролируя релизы и каждые внесённые изменения.
2. Хранилище – репозиторий, в нём хранятся все файлы проекта и все его версии.

commit – изменения внесённые в репозитории

история – это история изменений файлов проекта

рабочая копия – копия, сделанная из версии репозитория, с которой непосредственно работает сам разработчик

3. Централизованные системы контроля версий имеют один центральный репозиторий, с которым работают все разработчики. Примером является CVS, который является уже устаревшей системой. В децентрализованных системах же используется множество репозиториев одного проекта у каждого из разработчиков, при этом репозитории можно объединять брать из каждого только то, что нужно. Примером является знакомый нам Git
4. Создаётся репозиторий, и разрабатывается проект. При внесении изменений файлы отправляются на сервер
5. Разработчик клонирует репозиторий к себе на компьютер, и после внесения изменений выгружает их на сервер в качестве отдельной версии. После этого разработчики с более высокими правами могут, например, объединить его версию с текущей

6. Хранение файлов проекта, а также обеспечение командной работы, и контроль за версиями проекта
7. `git clone` – клонирует проект с сервера на компьютер `git add` – добавляет папку для выгрузки на сервер `git commit` – фиксирует изменения репозитория `git push` – выгружает изменения на сервер `git pull` – получить изменения с сервера `git rm` – удалить файл `git status` – статус репозитория
8. С локальным: `git commit -am "added files"` – создаёт коммит С удалённым: `git push` – загрузить данные на удалённый сервер
9. Ветки – это несколько независимых копий проекта, в каждой из которых ведётся разработка какой-то конкретной функции, при этом ветки существуют параллельно. Они нужны, когда нужно параллельно вести разработку нескольких функций, а в конце их можно объединить в одну
10. Игнорировать файлы можно, внося их в файл `.gitignore`. Игнорировать файлы нужно, когда их не нужно добавлять в репозиторий. Например, это могут быть файлы виртуального окружения (`venv`)

Список литературы

1. Kulyabov. Лабораторная работа № 3. Markdown. RUDN.